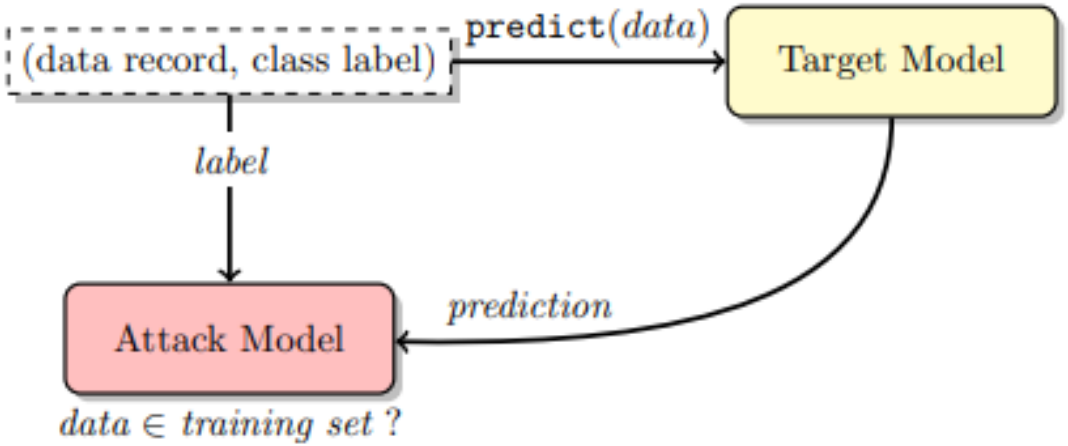
Membership Inference Attacks Against Machine Learning Models

Reza Shokri, Marco Stronati, Congzheng Song, Vitaly Shmatikov

Membership inference

- Given a machine learning model and a record, determine whether this record was used as part of the model's training dataset or not.



What we have?

- Training data's info:
  - Another part of training data. (1)
  - Training data's distribution / noisy training data. (2)
  - Statistic infos. (3)
  - Nothing. (4)

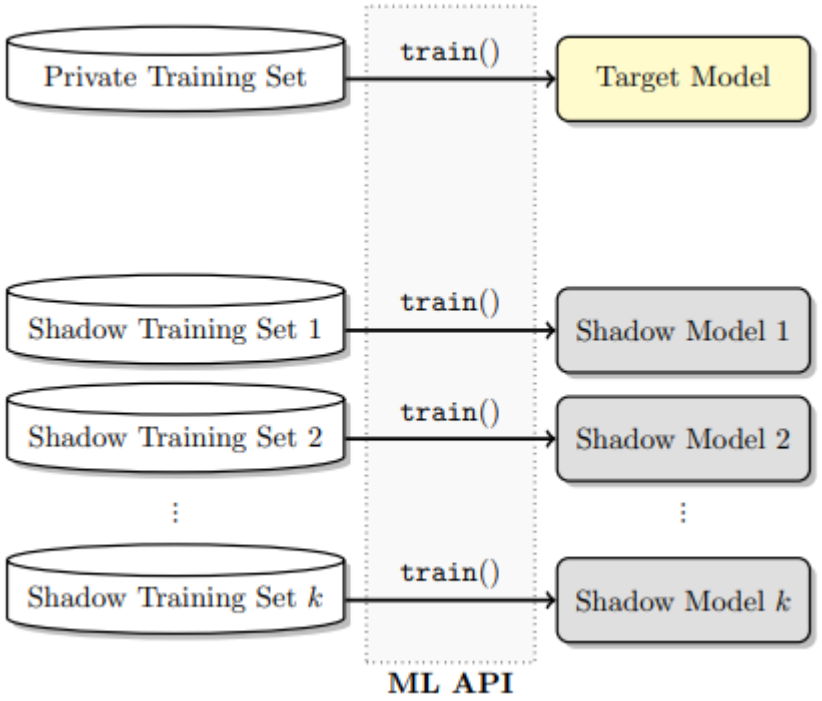
- Model's info:
  - Model's structure and training algorithm. (1)
  - Black-box query access to the target model. (2)

Key point: Mimic

- We purposed to mimic target model's behavior.
- **Shadow model** !!!

For each class i, we plan to train a shadow model $f^{i}_{shadow}()$. Each shadow model trained on the data distribution of class i and try to predict if one instance is in its training set (binary classification problem).

Why not just one shadow model ?



Shadow models' food

- Everything seems easy now…
- but how to obtain the training data of shadow model?
- We need to ensure the synthetic data should be from the same distribution of private training data.

- Purposed method:
  - Model-based synthesis
  - Statistics-based synthesis
  - Noisy real data

Model-based synthesis

- What we have:
  - Data aspect: nothing.
  - Model aspect: black-box query access (even can get logits!)

- Genetic algorithm…
  - After we query the model an img => the prediction label and its logits.
  - Walk an random step.
  - $y_c > current\ best$, Update.
    - $current\ best > acceptable\ threshold$, Return
  - Else keep random.

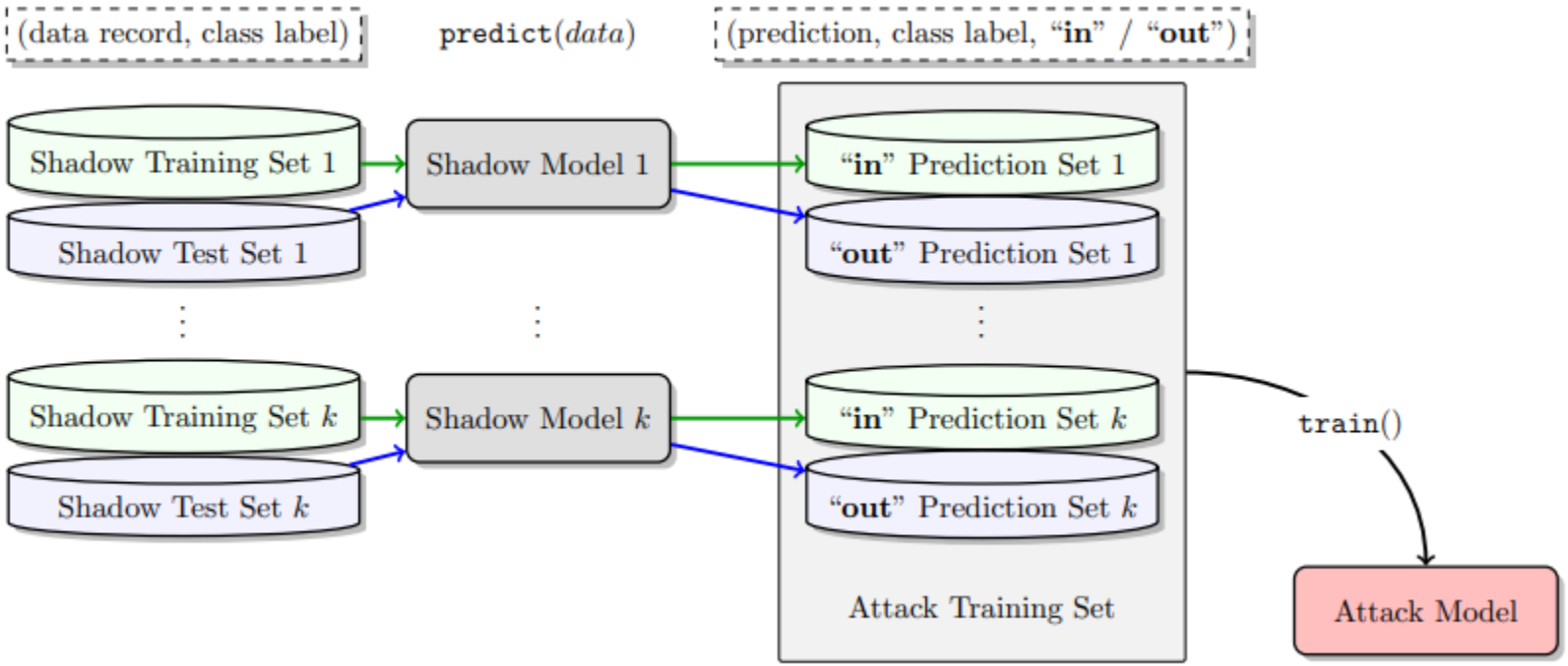
Framework

Algorithm 1 Data synthesis using the target model

```
1: procedure SYNTHESIZE(class : c)
2:     x ← RANDRECORD( )                    ▷ initialize a record randomly
3:     y*_c ← 0
4:     j ← 0
5:     k ← k_max
6:     for iteration = 1 ··· iter_max do
7:         y ← f_target(x)                  ▷ query the target model
8:         if y_c ≥ y*_c then               ▷ accept the record
9:             if y_c > conf_min and c = arg max(y) then
10:                if rand() < y_c then     ▷ sample
11:                    return x             ▷ synthetic data
12:                end if
13:            end if
14:            x* ← x
15:            y*_c ← y_c
16:            j ← 0
17:        else
18:            j ← j + 1
19:            if j > rej_max then          ▷ many consecutive rejects
20:                k ← max(k_min, ⌈k/2⌉)
21:                j ← 0
22:            end if
23:        end if
24:        x ← RANDRECORD(x*, k)            ▷ randomize k features
25:    end for
26:    return ⊥                             ▷ failed to synthesize
27: end procedure
```

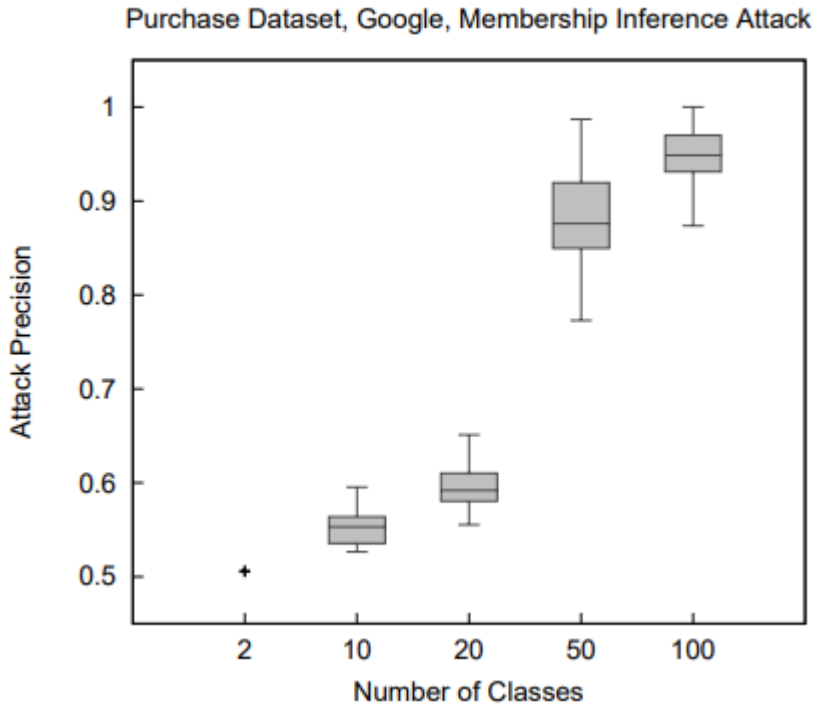
Why this works ???

Whole framework



Attack results

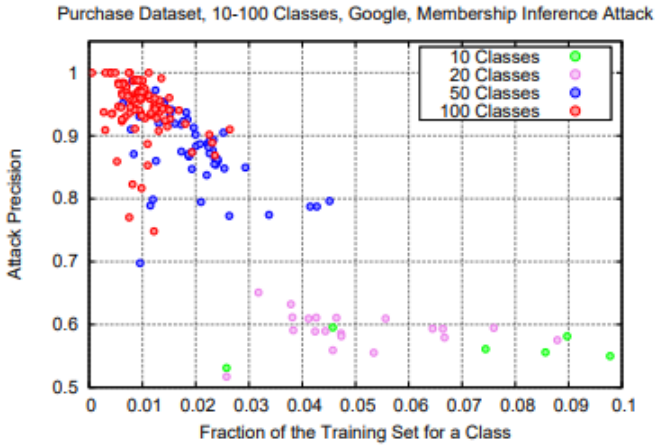
Dataset	*Training Accuracy*	*Testing Accuracy*	*Attack Precision*
Adult	0.848	0.842	0.503
MNIST	0.984	0.928	0.517
Location	1.000	0.673	0.678
Purchase (2)	0.999	0.984	0.505
Purchase (10)	0.999	0.866	0.550
Purchase (20)	1.000	0.781	0.590
Purchase (50)	1.000	0.693	0.860
Purchase (100)	0.999	0.659	0.935
TX hospital stays	0.668	0.517	0.657

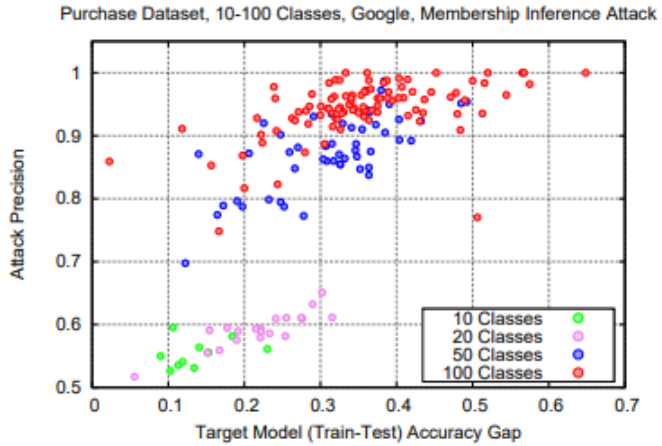


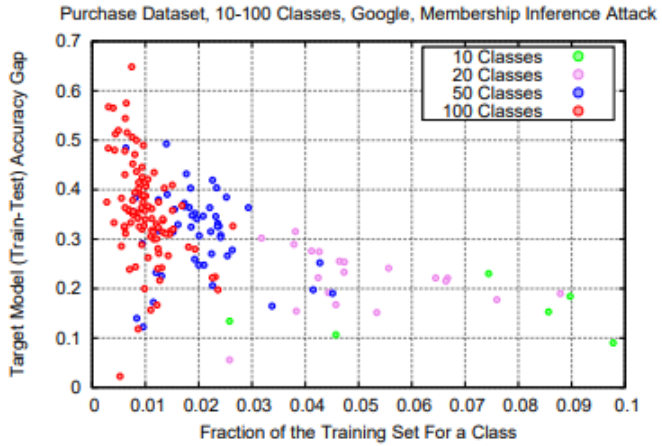
Other thoughts

- Overfitting => leaking data ?
  - “The more overfitted a model, the more it leaks—but only for models of the same type.”

- Bad generalization, non-diverse data => leaking data?







Privacy in Pharmacogenetics:
An End-to-End Case Study of Personalized Warfarin Dosing

Matthew Fredrikson*, Eric Lantz*, Somesh Jha*, Simon Lin†, David Page*, Thomas Ristenpart*

Model inversion

- Given a model trained to predict a specific variable, uses it to make predictions of unintended (sensitive) attributes used as input to the model (i.e., an attack on the privacy of attributes).

$f(x)$

age	height	weight	race	history	vkorc1	cyp2c9	dose
50-59	176.53	144.2	white				42.0
50-59	176.53	144.2	white				42.0
50-59	176.53	144.2	white				42.0

49.7	***p=0.23***
42.0	***p=0.75***
39.2	***p=0.01***

What we have?

- Training data's info:
  - Prior prob of prediction labels.
  - Target person's other public info.
  - Target person's prediction.

- Model's info:
  - Black-box query access.
  - Info about model's performance $\pi(y, y') = \mathbf{Pr}\left[\mathbf{z}_y = y | f(\mathbf{z}_\mathbf{x}) = y'\right]$

Case #1: Perfect model

Bayes rule:

$$\mathbf{Pr}\left[x_t|\mathbf{x}_K,y\right]=\frac{\mathbf{Pr}\left[x_t,\mathbf{x}_K,y\right]}{\mathbf{Pr}\left[\mathbf{x}_K,y\right]}=\frac{\sum_{\mathbf{x}'\in\hat{\mathbf{X}}:\mathbf{x}'_t=x_t}p(\mathbf{x}',y)}{\sum_{\mathbf{x}'\in\hat{\mathbf{X}}}p(\mathbf{x}',y)}$$

Maximum Entropy Principle:

$$p(\mathbf{x},y)=p(y)\cdot\prod_{1\leq i\leq d}p(\mathbf{x}_i)$$

MAP estimate:

$$\begin{aligned}\mathbf{Pr}\left[x_t|\mathbf{x}_K,y\right]&=\frac{\sum_{\mathbf{x}'\in\hat{\mathbf{X}}:\mathbf{x}'_t=x_t}p(y)\prod_i p(\mathbf{x}'_i)}{\sum_{\mathbf{x}'\in\hat{\mathbf{X}}}p(y)\prod_i p(\mathbf{x}'_i)}\\&\propto\sum_{\mathbf{x}'\in\hat{\mathbf{X}}:\mathbf{x}'_t=x_t}\prod_i p(\mathbf{x}'_i)\end{aligned}$$

Case #2: Imperfect model

Model's performance function:

$$\pi(y, y') = \mathbf{Pr}\left[\mathbf{z}_y = y | f(\mathbf{z}_\mathbf{x}) = y'\right]$$

$$\begin{aligned}
\mathbf{Pr}\left[x_t | \mathbf{x}_K, y^\alpha, f\right] &= \frac{\sum_{\mathbf{x}' \in \hat{\mathbf{X}}: \mathbf{x}'_t = x_t} \mathbf{Pr}\left[\mathbf{x}', y, f(\mathbf{x}')\right]}{\sum_{\mathbf{x}' \in \hat{\mathbf{X}}} \mathbf{Pr}\left[\mathbf{x}', y, f(\mathbf{x}')\right]} \\
&= \frac{\sum_{\mathbf{x}' \in \hat{\mathbf{X}}: \mathbf{x}'_t = x_t} \mathbf{Pr}\left[y | \mathbf{x}', f(\mathbf{x}')\right] p(\mathbf{x}')}{\sum_{\mathbf{x}' \in \hat{\mathbf{X}}} \mathbf{Pr}\left[\mathbf{x}', y, f(\mathbf{x}')\right]} \\
&\propto \sum_{\mathbf{x}' \in \hat{\mathbf{X}}: \mathbf{x}'_t = x_t} \pi_{y, f(\mathbf{x}')} \left(\prod_i p(\mathbf{x}'_i)\right)
\end{aligned}$$

Model Inversion Attacks that Exploit Confidence Information and Basic Countermeasures

Matt Fredrikson, Somesh Jha,Thomas Ristenpart

Drawbacks of Model inversion

- UNK features / Continuous features

- Computational infeasible while dim is large (image level)

How to achieve this?

- Gradient Descent!
- We started our attack image x from scratch.
- During every iteration:

$$\mathbf{x}_i \leftarrow \textsc{Process}(\mathbf{x}_{i-1} - \lambda \cdot \nabla c(\mathbf{x}_{i-1}))$$

- We keep following the gradient until meets the confidence threshold.

Detailed framework

Algorithm 1 Inversion attack for facial recognition models.

function MI-FACE($label, \alpha, \beta, \gamma, \lambda$)
  $c(\mathbf{x}) \stackrel{\text{def}}{=} 1 - \tilde{f}_{label}(\mathbf{x}) + \text{AUXTERM}(\mathbf{x})$
  $\mathbf{x}_0 \leftarrow \mathbf{0}$
  for $i \leftarrow 1 \dots \alpha$ **do**
    $\mathbf{x}_i \leftarrow \text{PROCESS}(\mathbf{x}_{i-1} - \lambda \cdot \nabla c(\mathbf{x}_{i-1}))$
    if $c(\mathbf{x}_i) \geq \max(c(\mathbf{x}_{i-1}), \dots, c(\mathbf{x}_{i-\beta}))$ **then**
      break
    if $c(\mathbf{x}_i) \leq \gamma$ **then**
      break
  return $[\arg\min_{\mathbf{x}_i}(c(\mathbf{x}_i)), \min_{\mathbf{x}_i}(c(\mathbf{x}_i))]$

Algorithm 2 Processing function for stacked DAE.

function PROCESS-DAE($\mathbf{x}$)
  encoder.DECODE($\mathbf{x}$)
  $\mathbf{x} \leftarrow$ NLMEANSDENOISE($\mathbf{x}$)
  $\mathbf{x} \leftarrow$ SHARPEN($\mathbf{x}$)
  return *encoder*.ENCODE(*vecx*)

- But… ?

Privacy attack => DP

A mechanism K achieves ε-differential privacy if for all databases D_1, D_2 differing in at most one row, and all $S \subseteq \text{Range}(\text{K})$,

$$\Pr[\text{K}(D_1) \in S] \leq \exp(\varepsilon) \times \Pr[\text{K}(D_2) \in S]$$

Differential Privacy can use raw private data as input, either output a new dataset with adding some perturbation or a trained model to achieve the differential privacy rules.